Detecting DDoS from the First Few Packets of a Flow (CIC-IDS2017)

SOP trial task, Track B (network security) | ID: 2023A7PS0646P | Submitted to Dr. K Hari Babu, BITS Pilani

1. Goal

The task was to train a simple classifier that separates attack flows from benign flows in CIC-IDS2017, with one restriction: it can only use information that a network switch could get from the first few packets of a flow. I report precision and recall for the attack class and then look at one case where the detector fails. Wherever randomness is involved (the train/test split, the CV folds and model training) I used seed 46, the last two digits of my ID.

The restriction comes from how a switch works. At line rate it can read header fields, match them against a small table and keep a few counters per flow, but it cannot wait for a flow to end or run a large model. So the real question is how far you can get with only what is visible during the TCP handshake.

2. Data and method

I used two days of the MachineLearningCVE CSVs, which have one row per flow (78 CICFlowMeter features plus a label). Friday afternoon has a DDoS attack generated with LOIC, and this is the day I trained on. Wednesday has four different DoS attacks (Hulk, GoldenEye, slowloris and Slowhttptest), which I only used to test the Friday models on unseen attacks. The official CIC server was not reachable, so I used a public GitHub LFS mirror and verified the files with SHA-256.

Column names had leading spaces, the rate columns (Flow Bytes/s, Flow Packets/s) had NaN or infinite values wherever the flow duration was zero, and there were many exact duplicates. I replaced inf with NaN, dropped those rows and removed duplicates before splitting, so that the same row cannot end up in both train and test. This left 223,082 Friday flows (128,014 DDoS, 95,068 benign) and 610,492 Wednesday flows (193,756 attacks). The label is 1 for anything that is not BENIGN. Friday was split 70/30, stratified by label.

I compared four feature sets:

- **EARLY**: Destination Port, Init_Win_bytes_forward, Init_Win_bytes_backward and min_seg_size_forward. All four are fixed by the SYN and SYN-ACK, so a switch has them within the first 1-3 packets. The initial window is chosen by the sender's TCP stack (8192 is common on Windows, 29200 on Linux), so it also says something about the sending machine.
- **EARLY_NO_PORT**: the same without the port, to see how much the port alone contributes.
- **COUNTER**: forward/backward packet totals, total forward bytes, largest forward packet and flow duration. A switch could keep these as per-flow registers, but in the CSV they are computed over the whole flow, so this set is an optimistic upper bound.
- **ALL**: all 78 features in a random forest with 100 trees, only as a reference ceiling.

For the first three I used a decision tree with max_depth = 5. I went with a shallow tree because every path from the root to a leaf is just a set of range checks on header fields, which maps directly onto match-action table entries in a programmable switch. Depth 5 means at most 32 entries.

3. Results

Table 1. Friday, held-out 30% (attack class).

Model	Precision	Recall	FP	FN	Leaves
EARLY	0.9896	0.9995	405	18	16
EARLY_NO_PORT	0.9451	0.9995	2,228	18	15
COUNTER	0.9995	0.9986	21	55	15
ALL (random forest)	1.0000	0.9999	1	5	–

The EARLY tree gets precision 0.9896 and recall 0.9995 with 16 leaves. 5-fold CV gives precision 0.9896 ± 0.0004 and recall 0.9998, so it is not a lucky split. Without the port the false positives go up from 405 to 2,228.

These numbers looked too good, so I checked what the tree was actually using. Init_Win_bytes_forward has an importance of 0.585 and the port 0.331. On Friday the DDoS flows use only two initial window values, 256 (54.5%) and 8192 (45.3%), all of them have min_seg_size_forward = 20, and all but 3 go to port 80. A fixed rule with no training at all, "window is 256 or 8192 and port is 80", already gets precision 0.9773 and recall 0.9983 on Friday. So the tree has mostly learned what LOIC's SYN packets look like, not what an attack looks like in general.

Table 2. Friday-trained models applied to Wednesday without retraining (the last four columns are the fraction of each attack's flows flagged).

Model	Precision	Recall	Hulk	GoldenEye	slowloris	Slowhttptest
EARLY	0.0855	0.0035	0.40%	0%	0%	0%
COUNTER	0.4603	0.0029	0.31%	0%	0%	0.63%
ALL (random forest)	0.5073	0.0007	0.08%	0%	0%	0.02%

On Wednesday every model fails (EARLY_NO_PORT behaves like EARLY). EARLY recall falls to 0.0035, and the models with more features are no better (ALL gets 0.0007) because they just fit Friday more closely. The fingerprint rule catches 3 of the 193,756 Wednesday attacks.

As a robustness check I retrained EARLY with seeds 0-9 as well. Friday precision stayed between 0.989 and 0.991, but Wednesday recall was always 0.0035 (seed 46), 0.3075 or 0.5724 (64% of Hulk). The difference comes from two small leaves that some seeds label as attack, for window = 251 and window = 0, built from only 17 and 27 Friday flows. Hulk's most common windows happen to be 0 (34%) and 251 (33%). So any cross-day success here is coincidence, and same-day metrics cannot tell the trees apart. I kept seed 46 as the main result.

4. Failure case: slowloris and Slowhttptest

None of the models flag a single slowloris flow, and the best one catches only 0.63% of Slowhttptest. Their first packets explain why. Their SYNs carry an initial window of 29200 (71.8% of slowloris and 80.6% of Slowhttptest flows) or 229 (28.0% and 18.5%). Both are ordinary values: on Friday, 229 is 27.5% of benign flows and 29200 is 6.3%. Their median min_seg_size_forward values (32 and 40) also show up in benign traffic. So their handshake looks like any normal Linux client.

The attacks are also low-rate by design: they open a connection and send partial HTTP headers very slowly so the server keeps the slot reserved. The packet and byte counters therefore look normal too, which is why COUNTER and ALL miss them. What actually separates them from benign traffic is timing: connections that stay open for a long time while carrying very little data, long idle gaps, and many half-open connections from the same source at once. None of this is visible until several seconds in, so a decision at packet 1-3 cannot catch it; a switch would need per-source state and timers.

5. Limitations

- The CSVs are per finished flow, so these are not real first-k-packet features (COUNTER uses whole-flow totals). Doing this properly needs the original pcaps.
- A random split within one day overstates performance, since the same attackers and tool are in both train and test, and on unseen attacks the result depends on the seed.
- CIC-IDS2017 has known labelling and CICFlowMeter issues (NaN/inf rows, 2,629 + 80,914 duplicates, a duplicated Fwd Header Length column).

Code, logs and exact commands: github.com/SAMARTHKOTHARI2005/sop-trackB-ddos. AI use: I used Claude (claude.ai, Claude Code) for planning, coding, parts of the analysis and drafting this report; details in report/AI_USE.md.